Last lecture: high-level overview of GPUs and performance
This lecture: benchmarking/profiling + writing kernels

> 上一讲：GPU 与性能的高层概览 本讲：基准测试/性能分析 + 编写 kernel

Summary:

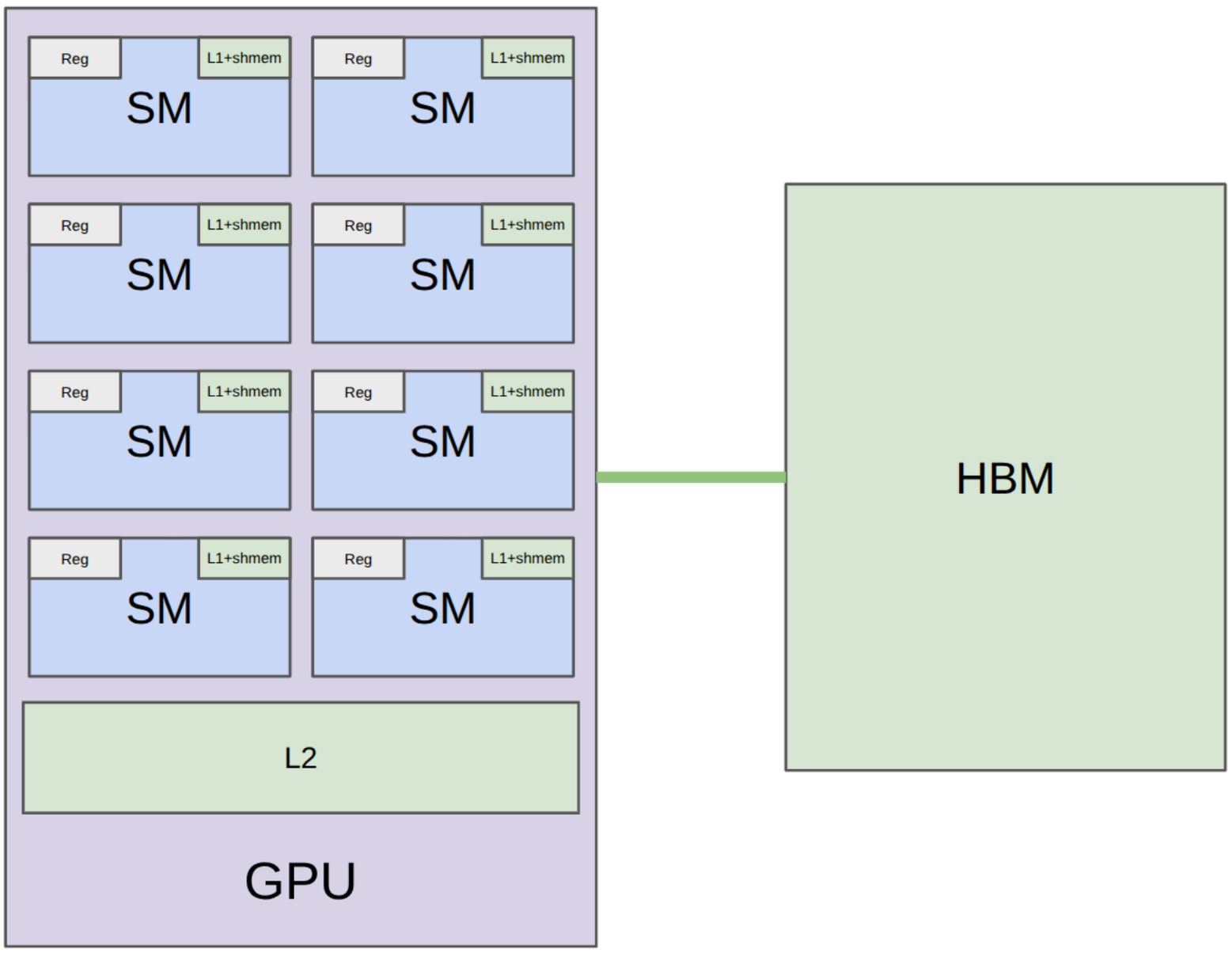
- Know the programming model (PyTorch, Triton, PTX) to give you correctness
- Understand the hardware (SMs, warps, occupancy, bank conflicts, etc.) to optimize performance
- Benchmark to understand scaling
- Profile to see what's being executed for how long
- Triton: think in terms of thread blocks (read to shared memory, do stuff (fusion), write back HBM)
- Examples: GeLU (elementwise), softmax (row-wise), row sum (baby tiling), matmul (tiling)

> 总结： 掌握编程模型（PyTorch、Triton、PTX）以保证正确性 理解硬件（SM、warp、occupancy、bank 冲突等）以优化性能 通过 benchmark 理解扩展规律 通过 profile 查看什么在执行、执行了多久 Triton：以 thread block 为单位思考（读入 shared memory，做计算（融合），写回 HBM） 示例：GeLU（逐元素）、softmax（按行）、行求和（入门级 tiling）、matmul（分块）

Next time: more than one GPU!

> 下一讲：不止一块 GPU！

Hardware



> 图：GPU 硬件结构示意：一块 GPU 内含 8 个 SM（4×2 排列），每个 SM 内有 8 份寄存器（Reg ×8）和 8 份 L1/共享内存（L1+shmem ×8）；SM 之下是统一的 L2 缓存，最底部为 HBM 显存。

Accelerator	A100	H100	B200
# SMs	108	132	148
Register size (per SM)	256 KB	256 KB	256 KB

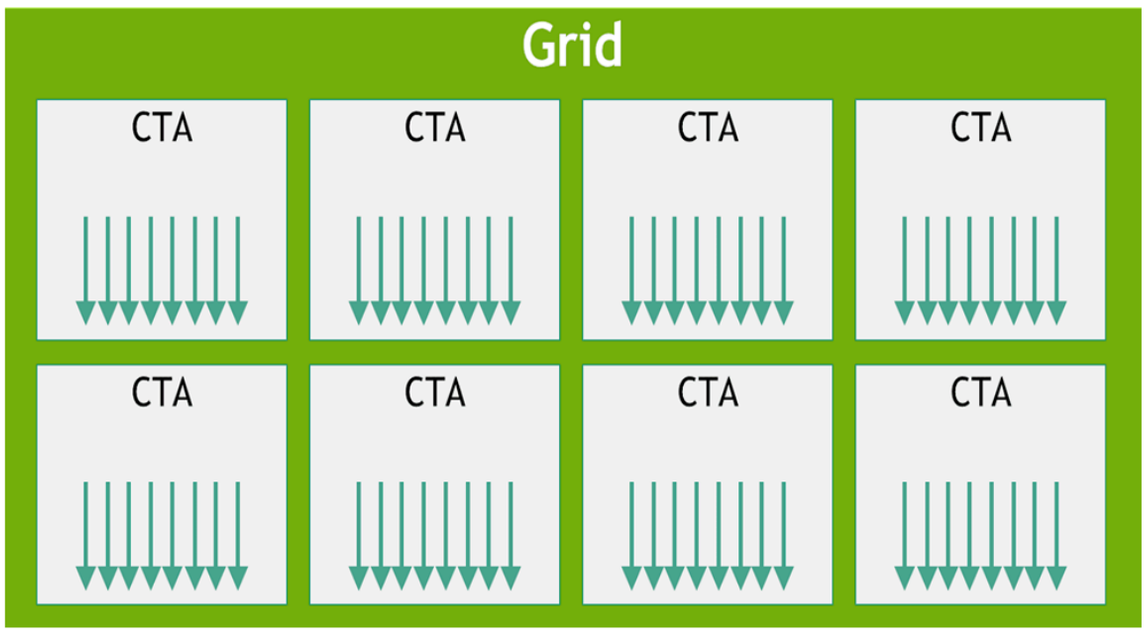
```
| L1 cache + shared memory (per SM)  |    192 KB |    256 KB |    256 KB |
| L2 cache size                      |     40 MB |     50 MB | 96-126 MB |
| HBM size                           |     80 GB |     80 GB |    192 GB |
+------------------------------------+-----------+-----------+-----------+
| Register bandwidth                 | ~116 TB/s | ~401 TB/s | ~447 TB/s |
| L1 cache + shared memory bandwidth |  ~19 TB/s |  ~33 TB/s |  ~40 TB/s |
| L2 cache bandwidth                 | ~5-8 TB/s |  ~12 TB/s |  ~17 TB/s |
| HBM bandwidth                      |    2 TB/s | 3.35 TB/s |    8 TB/s |
```

> 硬件 | 加速器 | A100 | H100 | B200 | | SM 数量 | 108 | 132 | 148 | | 寄存器大小（每 SM） | 256 KB | 256 KB | 256 KB | | L1 cache + shared memory（每 SM） | 192 KB | 256 KB | 256 KB | | L2 cache 大小 | 40 MB | 50 MB | 96-126 MB | | HBM 大小 | 80 GB | 80 GB | 192 GB | | 寄存器带宽 | ~116 TB/s | ~401 TB/s | ~447 TB/s | | L1 cache + shared memory 带宽 | ~19 TB/s | ~33 TB/s | ~40 TB/s | | L2 cache 带宽 | ~5-8 TB/s | ~12 TB/s | ~17 TB/s | | HBM 带宽 | 2 TB/s | 3.35 TB/s | 8 TB/s |

Note: the L1/L2 bandwidth numbers are rough estimates (online sources differ).
(B200s also have tensor memory (TMEM) for tensor cores (between registers and shared memory) that are invisible to programmer.)

> 注意：L1/L2 带宽数字只是粗略估计（不同网络来源并不一致）。（B200 还有供 tensor core 使用的 tensor memory（TMEM），位于寄存器和 shared memory 之间，对程序员不可见。）

Programming model



> 图：CUDA 编程模型示意图——由多个 CTA（协作线程阵列，即线程块）组成的线程块网格（grid），标注了 gridDim.x 与 blockDim.x，每个线程块内组织若干线程。

- *Thread*: executes code on a small part of the data
- *Thread block* or concurrent thread array (CTA): a group of threads
- *Grid*: collection of thread blocks

> 编程模型 线程（thread）：在数据的一小部分上执行代码 线程块（thread block）或并发线程数组（CTA）：一组线程 Grid：线程块的集合

(H100s and B200s also have thread block clusters that enable distributed shared memory.)

> （H100 和 B200 还有 thread block cluster，可以启用分布式 shared memory。）

Why thread blocks?
For elementwise operations (e.g., GeLU), threads are most natural: each thread processes one element.

- f(i) for i = 0, ..., N-1

However, for non-elementwise operations like softmax or matrix multiplication, threads need to communicate.
Reading/writing from HBM is slow, so use shared memory (local to SM).
Thread block: a collection of threads that access the same shared memory.
Consequently, a thread block is scheduled on one SM.

In Triton, think natively in terms of thread blocks (later).

> 为什么要有线程块？ 对逐元素操作（例如 GeLU）来说，线程是最自然的单位：每个线程处理一个元素。 然而，对于 softmax 或矩阵乘法这类非逐元素操作，线程之间需要通信。 读写 HBM 很慢，因此要使用 shared memory（SM 本地）。 线程块：访问同一块 shared memory 的线程集合。 因此，一个线程块会被调度到同一个 SM 上。 在 Triton 中，直接以线程块为单位来思考（稍后详述）。

Interaction between programming model and hardware

Programming model provides an abstraction of the hardware.
In principle, don't need to think about anything else (for correctness).
In practice, performance is very sensitive to the hardware, so need to understand it to obtain high performance.

> 编程模型与硬件之间的相互作用 编程模型提供了对硬件的抽象。 原则上（就正确性而言），你不需要考虑其他任何东西。 实践中，性能对硬件极其敏感，所以要想获得高性能就必须理解硬件。

Let's go over some considerations.

> 下面过一遍若干需要考虑的因素。

Warps:

- Within a thread block, threads are grouped into warps (32 threads per warp).
- Example: thread block has 64 threads => it has 2 warps.

```
| TTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTT | TTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTT |
```

- All threads within a warp must execute same instructions in lockstep on an SM.
- Control divergence: if different threads in a warp need to execute different instructions (if A, else B), must be done sequentially (bad)

```
| AAAAAAAAA.......................... |
| .........BBBBBBBBBBBBBBBBBBBBBBB |
```

- SM runs multiple warps and switches between them (e.g., when one warp is blocked on HBM reads/writes) with zero cost.

> Warp： 在线程块内部，线程被分成若干 warp（每个 warp 32 个线程）。 示例：线程块有 64 个线程 => 即 2 个 warp。 同一 warp 内的所有线程必须在同一个 SM 上以锁步（lockstep）方式执行相同的指令。 控制流分歧（control divergence）： 如果 warp 中不同的线程需要执行不同的指令（if A、else B），就必须串行执行（不好） SM 会同时运行多个 warp，并在它们之间零开销地切换（例如当某个 warp 阻塞在 HBM 读写上时）。

(Warp) occupancy:

- Each thread can use between 0 and 255 registers.
- The more registers threads use, the fewer threads can be scheduled on an SM (low occupancy).
- Low occupancy isn't necessarily bad if each thread is doing more work.
- Example: thread coarsening (each thread processes multiple elements).
- Example: thread block has 64 threads, each using 160 registers, SM has 65536 registers

> （warp）occupancy： 每个线程可以使用 0 到 255 个寄存器。 线程用的寄存器越多，能调度到一个 SM 上的线程就越少（低 occupancy）。 如果每个线程做的工作更多，低 occupancy 未必是坏事。 示例：thread coarsening（每个线程处理多个元素）。 示例：线程块有 64 个线程，每线程使用 160 个寄存器，SM 共有 65536 个寄存器

Bank conflicts (shared memory):

- Shared memory is divided into 32 banks, each 4 bytes wide.

```
B00 B01 B02 B03 B04 B05 B06 B07 B08 B09 B10 B11 B12 B13 B14 B15 B16 B17 B18 B19 B20 B21 B22 B23 B24 B25 B26 B27 B28 B29 B30
... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ...
... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ...
... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ... ...
```

- Each cycle, each bank can only be accessed by one thread (if not the same exact location).
- If multiple threads access the same bank, accesses serialized (bank conflict).
- Worst case example: matrix where each row spans all banks; 32 threads accessing first column results in 32-way bank conflict!
- Unavoidable: when doing matmul A @ B, access rows of A and columns of B

- Solution: swizzling rearranges shared memory (e.g., row xor col) to avoid bank conflicts

> Bank 冲突（shared memory）： shared memory 被划分为 32 个 bank，每个 bank 宽 4 字节。 每个周期，每个 bank 只能被一个线程访问（除非访问的是完全相同的地址）。 如果多个线程访问同一个 bank，访问会被串行化（bank 冲突）。 最坏情形示例：矩阵每一行横跨所有 bank；32 个线程访问第一列就会导致 32 路 bank 冲突！ 难以避免：做 matmul A @ B 时，要访问 A 的行和 B 的列 解决方案：swizzling 重排 shared memory 布局（例如 row xor col）以避免 bank 冲突

Memory coalescing (HBM):

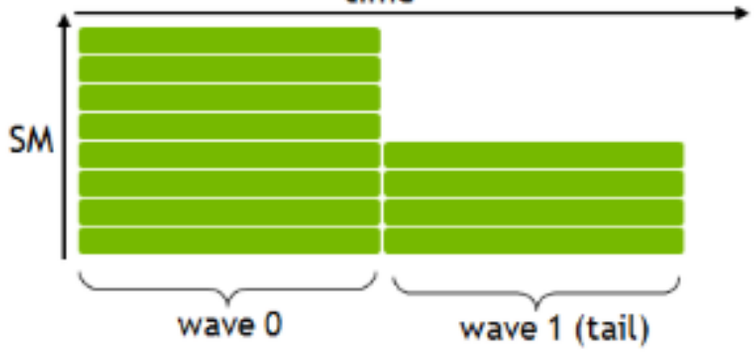
- When the 32 threads in a warp access HBM, memory accesses combined into transactions of 128 bytes (cache lines).

```
M00 M01 M02 M03 M04 M05 M06 M07 M08 M09 M10 M11 M12 M13 M14 M15 M16 M17 M18 M19 M20 M21 M22 M23 M24 M25 M26 M27 M28 M29 M30
M32 M33 M34 M35 M36 M37 M38 M39 M40 M41 M42 M43 M44 M45 M46 M47 M48 M49 M50 M51 M52 M53 M54 M55 M56 M57 M58 M59 M60 M61 M62
```

- Best case: full coalescing, all threads access the same cache line (32 threads x 4 bytes = 128 bytes).

> 访存合并（memory coalescing，HBM）： 当 warp 中的 32 个线程访问 HBM 时，访存请求会被合并成 128 字节的事务（cache line）。 最佳情形：完全合并，所有线程访问同一条 cache line（32 线程 x 4 字节 = 128 字节）。

Block occupancy:



> 图：GEMM 波数量化（wave quantization）示意图：纵轴为 SM，横轴为时间，wave 0 恰好占满所有 SM，wave 1 只剩一小段“尾部”（tail），造成算力浪费。

- Thread blocks scheduled onto SMs in waves.
- B200 has 148 SMs, if we launch 160 thread blocks, first wave has 148 blocks, second wave has 12 blocks.
- Wave quantization problem: last wave has fewer thread blocks, leaving some SMs idle (low block occupancy).
- Solution: make number of thread blocks divide # SMs.

> Block occupancy： 线程块以波（wave）为单位调度到各个 SM 上。 B200 有 148 个 SM，如果我们启动 160 个线程块，第一波有 148 个块，第二波只有 12 个块。 波量化（wave quantization）问题：最后一波的线程块更少，导致一些 SM 空闲（block occupancy 偏低）。 解决方案：让线程块的数量能被 SM 数量整除。

Summary:

- Programming model: grid (HBM) -> thread block (shared memory) -> thread (registers)
- Details of hardware (warps, bank conflicts, memory coalescing, occupancy) determine performance

> 总结： 编程模型：grid（HBM）-> thread block（shared memory）-> thread（寄存器） 硬件的细节（warp、bank 冲突、访存合并、occupancy）决定性能

Recipe for success:
1. Benchmark and profile your code
2. Make changes
3. Benchmark and profile your code again

> 成功秘诀： 对你的代码做 benchmark 和 profile 做出修改 再对你的代码做 benchmark 和 profile

Benchmark and profile your code!

> 一定要 benchmark 和 profile 你的代码！

Benchmarking measures the wall-clock time of performing some operation.
It only gives you end-to-end time, not where time is spent (profiling).

> 基准测试（benchmarking）测量执行某个操作的墙上时间。 它只给出端到端时间，不告诉你时间花在了哪里（那是 profiling 的事）。

It is still useful for:

- comparing different implementations (which is faster?), and
- understanding how performance scales (e.g., with dimension).

> 它仍然有助于： 比较不同的实现（哪个更快？），以及 理解性能如何扩展（例如随维度的变化）。

You can use `torch.utils.benchmark`.
We will roll our own to make benchmarking more transparent.

> 可以使用 torch.utils.benchmark。 我们会自己动手写一个，让基准测试更透明。

Note: time is roughly constant when dimension is small, then cubic scaling.

> 注意：维度较小时耗时大致恒定，之后按立方增长。

While benchmarking looks at end-to-end time, profiling looks at where time is spent.
Independent of time, profiling also helps you understand what's going under the hood.

> benchmarking 看的是端到端时间，而 profiling 看的是时间花在了哪里。 除了时间本身，profiling 还能帮助你理解底层到底发生了什么。

PyTorch has a built-in profiler.
In your assignment, you will use nsight to get more details.

> PyTorch 内置了 profiler。 在作业中，你们将使用 nsight 获取更多细节。

add(dim=2048)

```
                                                                                Name    Self CPU %      Self CP
---------------------------------------------------------------------------------------------------------------
void at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunctor_add, std::array<...         0.00%       0.000us
                                                                  Activity Buffer Request        97.09%       1.961m
                                                                        cudaLaunchKernel         2.19%       44.216u
                                                                    cudaDeviceSynchronize         0.72%       14.478u
---------------------------------------------------------------------------------------------------------------
Self CPU time total: 2.019msSelf CUDA time total: 4.833us
```

matmul(dim=2048)

Name	Self CPU %	Self CPU	CPU
cutlass3x_sm100_simt_sgemm_f32_f32_f32_f32_f32_64x64x16_1x1x1_3_nnn_align1_bias_f32_relu	0.00%	0.000us	
cuLaunchKernelEx	1.30%	24.828us	
Activity Buffer Request	97.93%	1.871ms	
cudaDeviceSynchronize	0.77%	14.746us	

Self CPU time total: 1.910msSelf CUDA time total: 330.366us

matmul(dim=128)

Name	Self CPU %	Self CPU	CPU
cutlass3x_sm100_simt_sgemm_f32_f32_f32_f32_f32_32x32x16_1x1x1_3_nnn_align1_bias_f32_relu	0.00%	0.000us	
cuLaunchKernelEx	1.48%	27.125us	
Activity Buffer Request	97.72%	1.795ms	
cudaDeviceSynchronize	0.81%	14.796us	

Self CPU time total: 1.837msSelf CUDA time total: 4.544us

Observations:

- You can see which CUDA kernels are actually being called (the long names).
- Different CUDA kernels are invoked depending on the tensor dimensions.

> 观察： 可以看到实际被调用的是哪些 CUDA kernel（那些很长的名字）。 根据张量维度的不同，会调用不同的 CUDA kernel。

Name of CUDA kernel tells us something about the implementation.
Example: cutlass3x_sm100_simt_sgemm_f32_f32_f32_f32_f32_64x64x16_1x1x1_3_nnn_align1_bi...

- cutlass: NVIDIA's CUDA library for linear algebra
- sm100: corresponds to the NVIDIA Blackwell architecture (B200)
- f32: float32

- 64x64x16: tile shape (more on this later)

> CUDA kernel 的名字能告诉我们一些关于实现的信息。 示例：
> cutlass3x_sm100_simt_sgemm_f32_f32_f32_f32_f32_64x64x16_1x1x1_3_nnn_align1_bi... cutlass：NVIDIA 的线性代数 CUDA 库 sm100：对应 NVIDIA Blackwell 架构（B200） f32：float32 64x64x16：tile 形状（稍后详述）

Let's benchmark and profile the GeLU activation function.

> 我们来对 GeLU 激活函数做 benchmark 和 profile。

The builtin and compiled versions are significantly faster!

> 内置版本和编译版本明显更快！

To understand why, let's look at the profiler to see where time is being spent.

> 为了理解原因，我们用 profiler 看看时间花在了哪里。

naive_gelu

```
                                                                                                                  Name    Self CPU %      Self CP
-------------------------------------------------------------------------------------------------------------------------------------------------
void at::native::vectorized_elementwise_kernel<4, at::native::BinaryFunctor<float, float, float, ...         0.00%       0
                                                                                                 Activity Buffer Request        24.65%       1.372m
                                                                                                       cudaLaunchKernel        23.63%       1.315m
                                                                                                 cudaDeviceSynchronize        51.72%       2.879m
-------------------------------------------------------------------------------------------------------------------------------------------------
Self CPU time total: 5.566msSelf CUDA time total: 3.384ms
```

builtin_gelu

```
                                                                                                                  Name    Self CPU %      Self CP
-------------------------------------------------------------------------------------------------------------------------------------------------
void at::native::vectorized_elementwise_kernel<4, at::native::GeluCUDAKernelImpl(at::TensorIterat...         0.00%       0
                                                                                                 Activity Buffer Request        84.27%       1.618m
                                                                                                       cudaLaunchKernel        15.15%     291.008u
                                                                                                 cudaDeviceSynchronize         0.57%      10.983u
-------------------------------------------------------------------------------------------------------------------------------------------------
Self CPU time total: 1.920msSelf CUDA time total: 304.640us
```

compiled_gelu

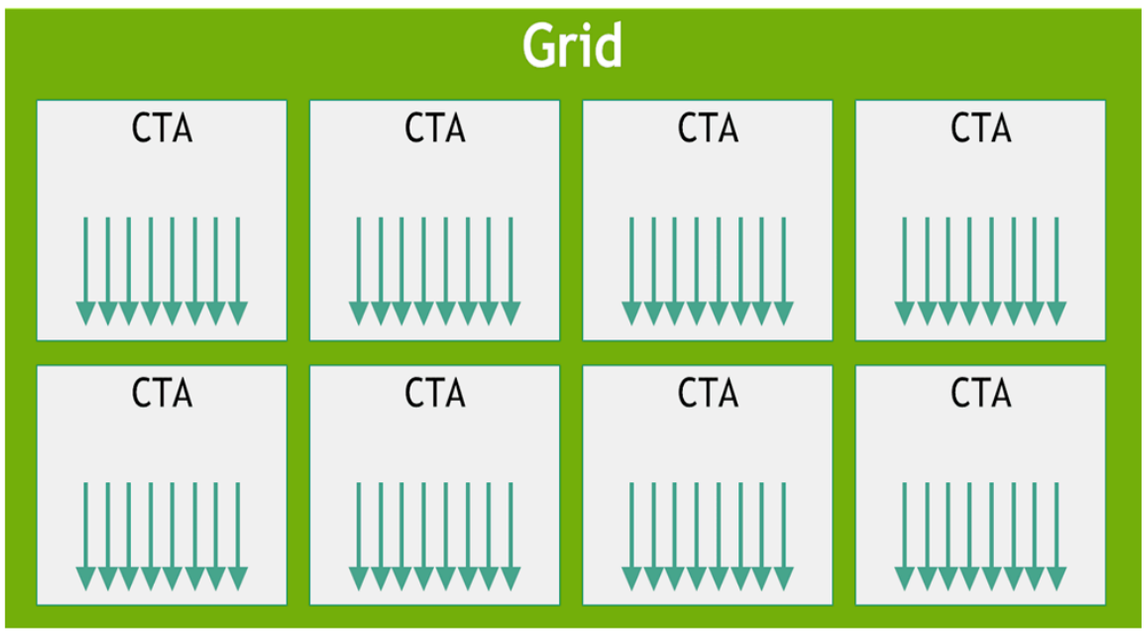
Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	Self CUDA	Self
triton_poi_fused_add_mul_tanh_0	0.00%	0.000us	0.00%	0.000us	0.000us	342.178us	1
Activity Buffer Request	87.65%	1.652ms	87.65%	1.652ms	1.652ms	0.000us	
cuLaunchKernel	11.77%	221.966us	11.77%	221.966us	221.966us	0.000us	
cudaDeviceSynchronize	0.58%	10.907us	0.58%	10.907us	5.454us	0.000us	

```
Self CPU time total: 1.885msSelf CUDA time total: 342.178us
```

Notes:

- Naive implementation: multiple kernels, requires many reads/writes from/to HBM (**no fusion**).
- Builtin and compiled versions: one kernel (**kernel fusion**), one read from HBM, one write to HBM.
- The compiled kernel is a Triton kernel.

> 说明： 朴素实现：多个 kernel，需要对 HBM 进行大量读写（没有融合）。 内置版本与编译版本：单个 kernel（kernel 融合），从 HBM 读一次、向 HBM 写一次。 编译出来的 kernel 是一个 Triton kernel。



> 图：CUDA 线程组织层次：一个 Grid（网格）由多个 CTA（线程块，如 block(0,0) 至 block(3,2)）组成；局部放大显示 block(1,1) 内部 thread(0,0)–thread(2,2) 的线程排列。

In CUDA (developed by NVIDIA), specify what each thread does.

- Pros: fine-grained control
- Cons: need to manage more things (e.g., shared memory)

> 在 CUDA（NVIDIA 开发）中，你指定每个线程做什么。 优点：细粒度控制 缺点：需要管理更多的东西（例如 shared memory）

In Triton (developed by OpenAI), specify what each thread block does.

- Generally powerful enough (especially when getting started)
- Conceptual framework: load data into shared memory, operate on it, write back to global memory

> 在 Triton（OpenAI 开发）中，你指定每个线程块做什么。 通常已经足够强大（尤其是入门阶段） 概念框架：把数据加载到 shared memory，在其上做计算，再写回全局内存

Let's write the Triton kernel for GeLU.

> 我们来写 GeLU 的 Triton kernel。

Triton compiles down to PTX (parallel thread execution), an assembly language for GPUs.

> Triton 会编译到 PTX（parallel thread execution），一种 GPU 汇编语言。

We can see the PTX code generated by Triton.
[https://github.com/stanford-cs336/lectures/blob/main/var/triton_gelu-ptx.txt]

> 我们可以看到 Triton 生成的 PTX 代码。

Observations:

- ld.global.* and st.global.* reads and writes from global memory
- %ctaid.x is block index, %tid.x is thread index
- %f* are floating point registers, %r* are integer registers
- One thread processes 8 elements at the same time (thread coarsening)

> 观察： ld.global.* 与 st.global.* 是对全局内存的读和写 %ctaid.x 是 block 索引，%tid.x 是线程索引 %f* 是浮点寄存器，%r* 是整数寄存器 一个线程同时处理 8 个元素（thread coarsening）

So far, we've looked at elementwise operations in Triton (e.g., GeLU).
Now let us look at operations that aggregate over multiple values.

> 到目前为止，我们在 Triton 中看的是逐元素操作（例如 GeLU）。 现在来看对多个值做聚合的操作。

We will roughly follow the Triton fused softmax tutorial: [https://triton-lang.org/main/getting-started/tutorials/02-fused-softmax.html]

> 我们将大致跟随 Triton 的 fused softmax 教程：[https://triton-lang.org/main/getting-started/tutorials/02-fused-softmax.html]

Recall the softmax operation is used in attention and generating probabilities.
Exponentiate and normalize each row of a matrix:

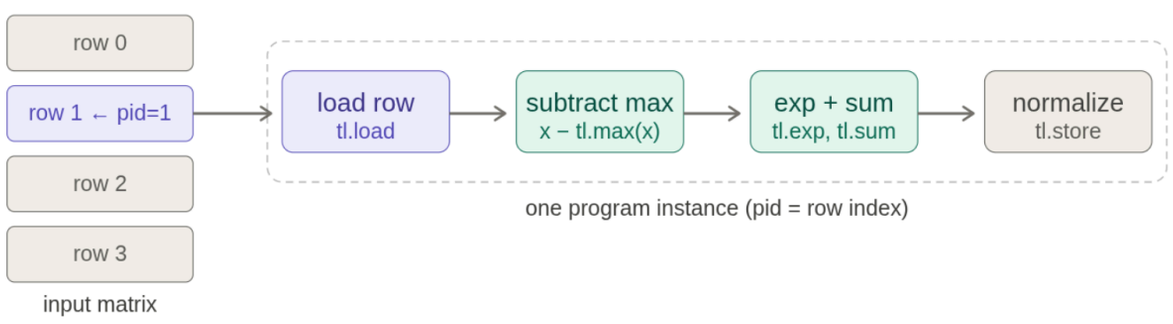
```
[0 0 0]       =>   [1/3 1/3 1/3]
[1 1 -inf]         [1/2 1/2 0   ]
```

> 回顾一下，softmax 用于 attention 以及生成概率。 对矩阵的每一行做指数运算并归一化：

Let's first start with the naive implementation and keep track of reads/writes.

> 我们先从朴素实现开始，并统计读写次数。

Now let us write the Triton kernel.



> 图：Triton 实现 softmax 的示意：输入矩阵的每一行（row 0–3）交给一个程序实例处理（pid 即行号，如 pid=1 处理 row 1），依次执行：tl.load 读入整行、减去最大值 x − tl.max(x)、tl.exp 求幂并 tl.sum 求和、归一化后 tl.store 写回。

> 现在来写 Triton kernel。

In the softmax example, an entire row fits in a block, so the reduction happens within a block (handled by Triton).
What if the row doesn't fit in a block?
Example: 4096 columns, but block size is 1024...

> 在 softmax 的例子中，一整行能装进一个 block，因此规约发生在 block 内部（由 Triton 处理）。 如果一行装不进一个 block 怎么办？ 示例：4096 列，但 block 大小是 1024.....

Strategy:

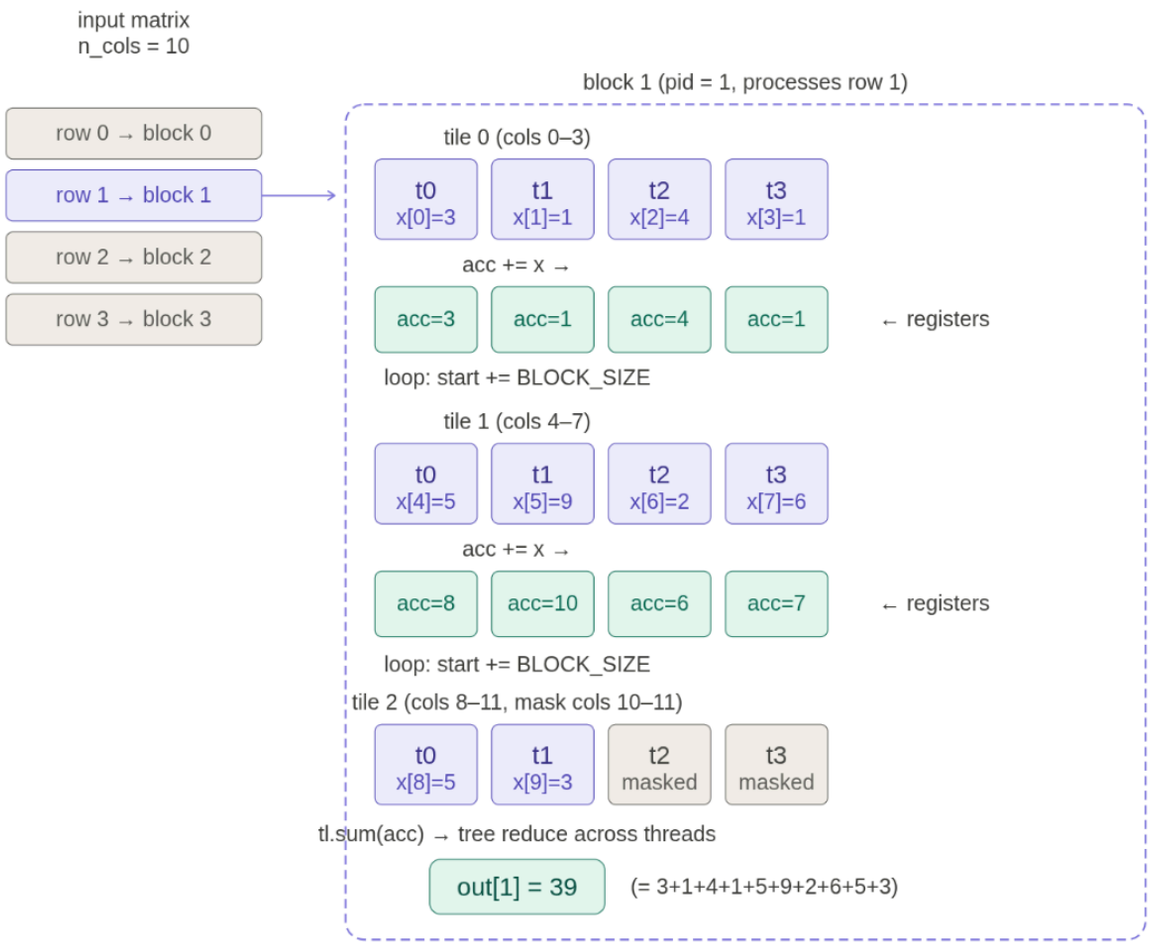
- Break up row into tiles (4 in the example above)

- Each thread iterates over tiles and accumulates a sum
- Do final reduction (sum) over accumulators of each thread (shared memory or warp shuffles)

> 策略： 把行拆成若干 tile（上例中为 4 个） 每个线程遍历各 tile 并累加求和 对各线程的累加器做最终规约（求和）（借助 shared memory 或 warp shuffle）

Consider the simpler example (row sum instead of softmax):

> 考虑一个更简单的例子（用行求和代替 softmax）：



> 图：行求和（row sum）的张量示意图——二维张量的行/列分别对应 axis 0（B 维）与 axis 1（D 维），沿 axis 1 规约（reduction along axis=1）得到每行的和，每行由一个 program 处理。

Matrix multiplication is the bread and butter of deep learning.

> 矩阵乘法是深度学习的看家本领。

How should we build a matmul kernel?

> 我们该如何构建一个 matmul kernel?

```
|          k                    n
|    [ A1 A2 A3 ]        [ B1 B2 B3 ]    [ C1 C2 C3 ]
| m  [ A4 A5 A6 ]   *   k [ B4 B5 B6 ] =  [ C4 C5 C6 ]
|    [ A7 A8 A9 ]        [ B7 B8 B9 ]    [ C7 C8 C9 ]
```

Naive approach:
Fix any (m, n).
For each k:

- Read A[m, k] and B[k, n] from HBM.
- Multiply and accumulate.

Write result to C[m, n] in HBM.

> 朴素做法： 固定任意 (m, n)。 对每个 k： 从 HBM 读取 A[m, k] 和 B[k, n]。 相乘并累加。 把结果写入 HBM 中的 C[m, n]。

Bottleneck: M K N reads, M N writes
Arithmetic intensity: O(1)

> 瓶颈：M K N 次读，M N 次写 算术强度：O(1)

Computing C4 and C5 both need A4, A5, A6.
Can we read A4, A5, A6 from HBM once to compute both?

Answer: yes, using shared memory!

> 计算 C4 和 C5 都需要 A4、A5、A6。 能不能只从 HBM 读一次 A4、A5、A6，就把两个结果都算出来？ 答案：可以，用 shared memory！

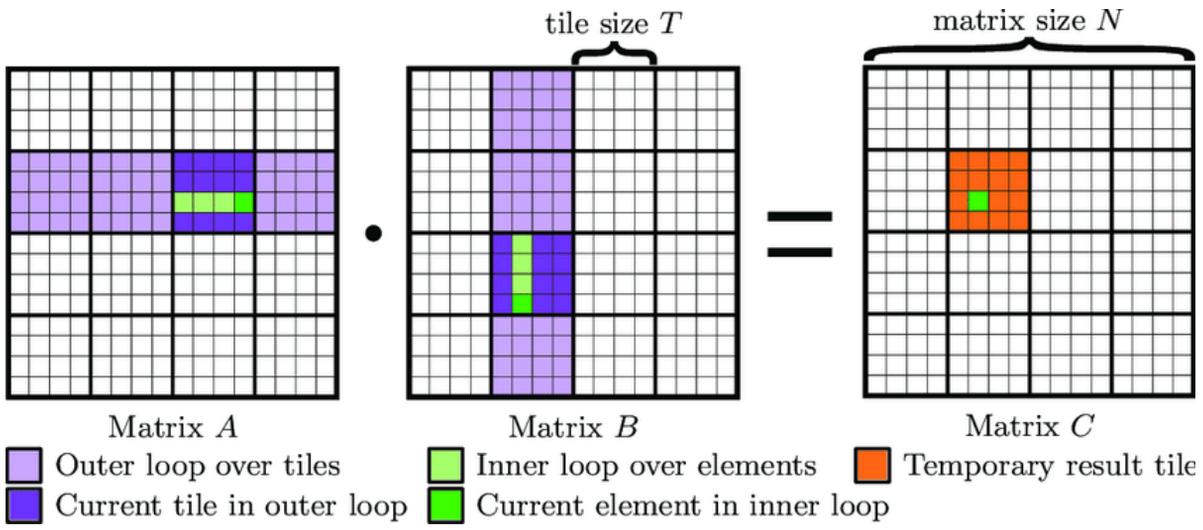
Idealized approach:

- Load all of A and B into shared memory, then compute C.
- Now we get M K + K N reads and M N writes.
- This yields the idealized O(N) arithmetic intensity from before.
- However, A and B are usually too large to fit in shared memory.

> 理想化做法： 把 A 和 B 全部加载进 shared memory，然后计算 C。 现在只需 M K + K N 次读和 M N 次写。 这就得到了之前所说的理想化 O(N) 算术强度。 然而，A 和 B 通常太大，放不进 shared memory。

Tiling:

> 分块（tiling）：



> 图：面向 Tensor Core 的分块（Tiling for Tensor Cores）：把矩阵乘切成 A、B、C 的 tile，数据从 HBM（300 GB/s、大容量）搬到 SRAM（19 TB/s、小容量）再进寄存器，逐块完成乘加。

Key idea: divide the matrix C into output tiles (thread blocks).
Fix an output tile in C.
For each pair of (row tile of A, column tile of B):

- Load the corresponding A tile and B tile from HBM into shared memory.
- Perform matrix multiplication on the tiles.
- Accumulate into the partial sum (in shared memory).

Write output tile to HBM.

> 关键思想：把矩阵 C 划分成若干输出 tile（线程块）。 固定 C 中的一个输出 tile。 对每一对（A 的行 tile，B 的列 tile）：把对应的 A tile 和 B tile 从 HBM 加载到 shared memory。 在这些 tile 上执行矩阵乘法。 累加到部分和里（存于 shared memory）。 把输出 tile 写回 HBM。

Arithmetic intensity: O(tile_size).

> 算术强度：O(tile_size)。

Bonus:

- Often, you want to apply an elementwise activation function.
- Example: GeLU(A @ B)
- Solution: kernel fusion!

> 附加内容： 很多时候，你还想再应用一个逐元素激活函数。 示例：GeLU(A @ B) 解决方案：kernel 融合！

Implementation.

> 实现。

Review: each matrix is linearized in memory

> 复习：每个矩阵在内存中都是线性化存储的